
Especificação Técnica

API Financeira Pro — Serviço REST de cotações, proventos e carteiras de acompanhamento

Projeto	<code>api-financeira</code>
Versão da API	1.0.0 (pacote <code>0.1.0</code>)
Especificação	OpenAPI 3.1 / REST
Linguagem	Python 3.12.0 (requer <code>>=3.12</code>)
Framework	FastAPI 0.133.1 sobre Starlette 0.52.1
Banco de dados	PostgreSQL 12+ — base <code>easy_finace</code>
Cache	Redis (imagem <code>redis:alpine</code>)
Autor	Felipe Nilo M. de Faria — github.com/fenilonilo
Documento	13 de agosto de 2026

1. Visão geral

A **API Financeira Pro** é um serviço HTTP REST assíncrono que expõe dados de ativos financeiros — ações brasileiras (B3), *stocks* americanas e criptomoedas — consumidos do Yahoo Finance, além de gerenciar contas de usuário e listas de acompanhamento (*watchlists*) persistidas em PostgreSQL.

Capacidades funcionais:

- **Search** — busca global de ativos na base do Yahoo Finance.
- **Quote** — preço e tendência (subindo / caindo / estável) em tempo quase real.
- **History** — série histórica de fechamento para construção de gráficos.
- **Financials** — múltiplos e indicadores fundamentalistas.
- **Dividends** — histórico dos 10 proventos mais recentes.
- **News** — as 5 notícias mais recentes, com resumo e fonte.
- **Auth / User / Profile** — cadastro, login JWT e watchlist por usuário.

Todas as respostas trafegam em `application/json`. A documentação interativa é gerada automaticamente em `/docs` (Swagger UI) e `/redoc`, com o contrato em `/openapi.json`.

2. Stack tecnológico e versões

2.1 Runtime e plataforma

Item	Versão	Observação
Linguagem	Python 3.12.0	Ambiente local (<code>.venv</code> , CPython gerenciado pelo uv)
Restrição declarada	<code>requires-python >= 3.12</code>	Definida em <code>pyproject.toml</code>
Python em container	3.11-slim	Imagem base do <code>Dockerfile</code> — divergente do ambiente local
Gerenciador de pacotes	uv 0.8.22	<code>pyproject.toml</code> + <code>uv.lock</code> (lockfile determinístico)
Servidor ASGI	Uvicorn 0.41.0	<code>uvicorn main:app --host 0.0.0.0 --port 8000</code>
Sistema operacional alvo	Linux (container) / Windows (dev)	Sem dependência de SO no código

2.2 Dependências diretas

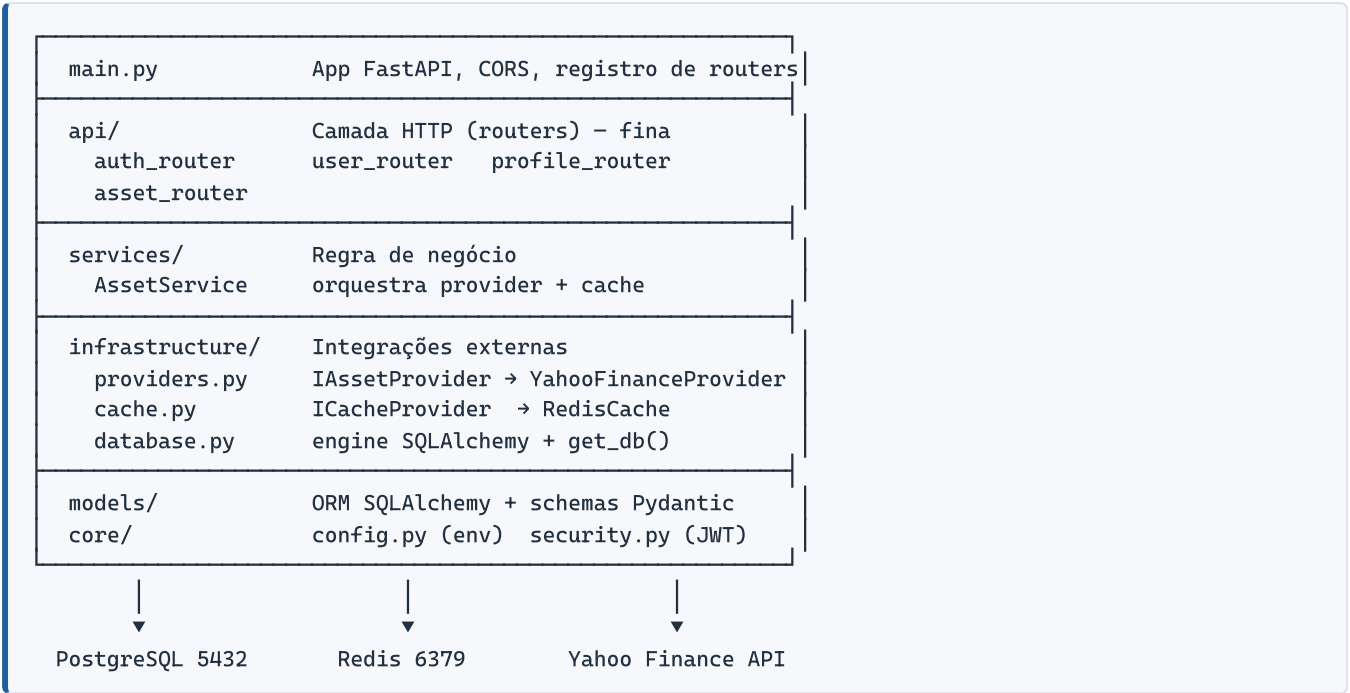
Pacote	Versão fixada	Restrição	Função
<code>fastapi</code>	0.133.1	livre	Framework web / roteamento / OpenAPI
<code>uvicorn</code>	0.41.0	livre	Servidor ASGI
<code>pydantic[email]</code>	2.12.5	livre	Validação de schemas; <code>EmailStr</code>
<code>sqlalchemy</code>	2.0.47	<code>>=2.0.47</code>	ORM e engine de banco
<code>psycopg2-binary</code>	2.9.11	<code>>=2.9.11</code>	Driver PostgreSQL (síncrono)
<code>redis</code>	7.2.1	livre	Cliente Redis (<code>redis.asyncio</code>)
<code>yfinance</code>	1.2.0	livre	Provedor de dados de mercado
<code>httpx</code>	0.28.1	livre	Cliente HTTP assíncrono (busca global Yahoo)
<code>pyjwt</code>	2.11.0	livre	Emissão e verificação de JWT
<code>passlib[bcrypt]</code>	1.7.4	<code>==1.7.4</code>	Hash de senha
<code>bcrypt</code>	4.0.1	<code>==4.0.1</code>	Backend do passlib (fixado por incompatibilidade com 4.1+)
<code>python-dotenv</code>	1.2.1	<code>>=1.2.1</code>	Carga do arquivo <code>.env</code>
<code>python-multipart</code>	0.0.22	<code>>=0.0.22</code>	Parsing de <code>form-data</code> no login OAuth2

2.3 Dependências transitivas relevantes

Pacote	Versão	Papel
<code>starlette</code>	0.52.1	Camada ASGI do FastAPI (middleware CORS, threadpool)
<code>anyio</code>	4.12.1	Concorrência estruturada; base do <code>run_in_threadpool</code>
<code>pandas</code>	3.0.1	<code>DataFrame</code> / <code>Series</code> retornados pelo yfinance
<code>numpy</code>	2.4.2	Base numérica do pandas
<code>curl-cffi</code>	0.13.0	Sessão HTTP usada internamente pelo yfinance

3. Arquitetura

Arquitetura em camadas com inversão de dependência: a camada HTTP não conhece o yfinance nem o Redis diretamente, apenas as interfaces `IAssetProvider` e `ICacheProvider`.



3.1 Padrões de projeto aplicados

Padrão	Onde	Efeito
Repository / Provider (ABC)	infrastructure/providers.py	Trocar a fonte de dados exige apenas nova implementação de IAssetProvider e ajuste em get_asset_service()
Strategy de cache (ABC)	infrastructure/cache.py	ICacheProvider abstrai o Redis; permite cache em memória para testes
Injeção de dependência	Depends() do FastAPI	get_db , get_current_user , get_asset_service
Cache-aside (read-through)	AssetService._get_with_cache	Consulta o Redis, cai para o provider em miss e regrava com TTL
Offload de I/O bloqueante	run_in_executor / run_in_threadpool	yfinance é síncrono; as chamadas saem do event loop
Fan-out concorrente	asyncio.gather em search_assets	Busca de ícones em paralelo: custo do mais lento, não da soma

3.2 Estrutura de diretórios

API_Financeira/	
├── main.py	App FastAPI + CORS + routers
├── pyproject.toml	Metadados e dependências (uv)
├── uv.lock	Lockfile determinístico
├── Dockerfile	Imagem da API (python:3.11-slim + uv)
├── docker-compose.yaml	API + Redis, rede rede_financeira
├── openapi.json	Contrato OpenAPI exportado
├── .env / .env.example	Variáveis de ambiente
├── api/	
│ ├── auth_router.py	POST /auth/login
│ ├── user_router.py	Registro, update, delete
│ ├── profile_router.py	Watchlist do usuário
│ └── asset_router.py	Endpoints de ativos
├── services/	
│ └── asset_service.py	AssetService (cache + provider + ícones)
├── infrastructure/	
│ ├── providers.py	IAssetProvider, YahooFinanceProvider
│ ├── cache.py	ICacheProvider, RedisCache
│ └── database.py	Engine, SessionLocal, Base, get_db
├── models/	
│ ├── user.py	User, UserWatchlist (ORM) + schemas
│ └── asset.py	Schemas Pydantic de ativos
└── core/	
│ ├── config.py	SECRET_KEY, ALGORITHM, REDIS_URL, TTL
│ └── security.py	Hash, JWT, get_current_user

4. Banco de dados

4.1 Configuração

Parâmetro	Valor
SGBD	PostgreSQL (recursos usados exigem 9.4+ para <code>JSONB</code> ; recomendado 14+)
Driver	<code>psycopg2-binary</code> 2.9.11 — conexão síncrona
ORM	SQLAlchemy 2.0.47 (<code>declarative_base</code> , <code>sessionmaker</code>)
Host / porta	<code>host.docker.internal:5432</code>
Base	<code>easy_finace</code>
Usuário	<code>postgres</code>
URL de conexão	<code>postgresql://postgres:***@host.docker.internal:5432/easy_finace</code>
Sessão	<code>autocommit=False</code> , <code>autoflush=False</code> ; encerrada no <code>finally</code> de <code>get_db()</code>
Pool	<code>QueuePool</code> padrão do SQLAlchemy (5 conexões + 10 de overflow)
Migrações	Não há ferramenta de migração (Alembic ausente); schema criado manualmente

Atenção: a URL de conexão está *hardcoded* em `infrastructure/database.py` , incluindo credenciais, e não é lida de variável de ambiente. Recomenda-se migrar para `DATABASE_URL` no `.env` .

4.2 Tabela `users`

Coluna	Tipo	Nulo	Regra
<code>id</code>	<code>uuid</code>	NOT NULL	PK; default <code>uuid4()</code> gerado na aplicação
<code>name</code>	<code>varchar(255)</code>	NOT NULL	Nome do usuário
<code>email</code>	<code>varchar(255)</code>	NOT NULL	<code>UNIQUE</code> + índice; validado como <code>EmailStr</code>
<code>password_hash</code>	<code>varchar(255)</code>	NOT NULL	Hash bcrypt; nunca retornado pela API
<code>birth_date</code>	<code>date</code>	NOT NULL	Data de nascimento
<code>investor_profile</code>	<code>varchar(20)</code>	NOT NULL	<code>CHECK</code> em <code>CONSERVATIVE MODERATE AGGRESSIVE</code>
<code>is_active</code>	<code>boolean</code>	NULL	Default <code>true</code> (aplicação)
<code>created_at</code>	<code>timestampz</code>	NULL	<code>server_default now()</code>
<code>updated_at</code>	<code>timestampz</code>	NULL	<code>server_default now()</code> , <code>onupdate now()</code> (aplicação)

Constraint nomeada: `users_investor_profile_check`.

4.3 Tabela `user_watchlist`

Coluna	Tipo	Nulo	Regra
<code>id</code>	<code>uuid</code>	NOT NULL	PK; default <code>uuid4()</code>
<code>user_id</code>	<code>uuid</code>	NOT NULL	FK → <code>users.id</code> ON DELETE CASCADE ; <code>UNIQUE</code> (1:1)
<code>tickers</code>	<code>jsonb</code>	NOT NULL	Array de objetos <code>{ticker, name, icon_url}</code> ; default <code>[]</code>
<code>added_at</code>	<code>timestampz</code>	NULL	<code>server_default now()</code>

Mutações no campo `tickers` exigem `flag_modified(watchlist, "tickers")` — o SQLAlchemy não detecta alteração *in-place* de estrutura JSONB.

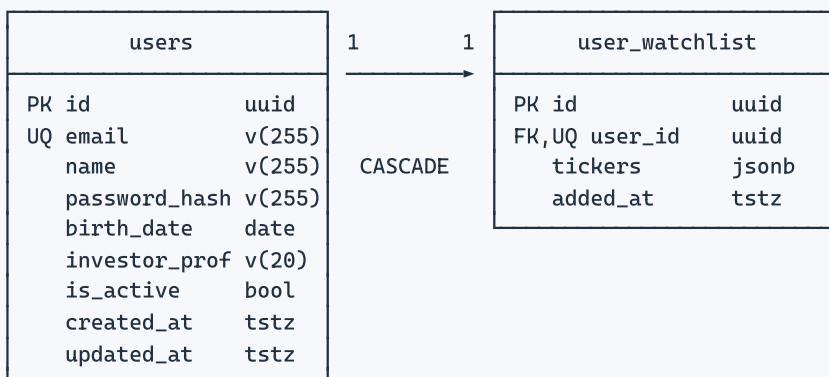
4.4 DDL equivalente

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

CREATE TABLE users (
  id          uuid          PRIMARY KEY DEFAULT uuid_generate_v4(),
  name        varchar(255)  NOT NULL,
  email       varchar(255)  NOT NULL UNIQUE,
  password_hash varchar(255) NOT NULL,
  birth_date  date          NOT NULL,
  investor_profile varchar(20) NOT NULL,
  is_active   boolean       DEFAULT true,
  created_at  timestampz    DEFAULT CURRENT_TIMESTAMP,
  updated_at  timestampz    DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT users_investor_profile_check
    CHECK (investor_profile IN ('CONSERVATIVE', 'MODERATE', 'AGGRESSIVE'))
);
CREATE INDEX ix_users_email ON users (email);

CREATE TABLE user_watchlist (
  id          uuid          PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id     uuid          NOT NULL UNIQUE
    REFERENCES users(id) ON DELETE CASCADE,
  tickers     jsonb         NOT NULL DEFAULT '[]'::jsonb,
  added_at    timestampz    DEFAULT CURRENT_TIMESTAMP
);
```

4.5 Modelo lógico



5. Camada de cache (Redis)

Cache de leitura no padrão *cache-aside*, acessado pelo cliente assíncrono `redis.asyncio` com `decode_responses=True`. Valores armazenados como JSON serializado; escrita via `SETEX` (TTL atômico).

Operação	Padrão de chave	TTL	Justificativa
Cotação	<code>quote_{TICKER}</code>	60 s	Default de <code>CACHE_TTL_SECONDS</code>
Busca	<code>search_{query default}</code>	3600 s	Catálogo muda pouco
Histórico	<code>hist_{ticker}_{period}</code>	3600 s	Série diária
Fundamentos	<code>fin_{ticker}</code>	86400 s	Balanços têm baixa frequência
Dividendos	<code>div_{ticker}</code>	86400 s	Proventos são esparsos
Notícias	<code>news_{ticker}</code>	1800 s	Equilíbrio entre frescor e custo

Sem persistência configurada (`redis:alpine` padrão) — a perda do cache não afeta a integridade dos dados, apenas a latência do primeiro acesso.

6. Autenticação e segurança

Mecanismo	Especificação
Esquema	OAuth2 Password Flow (<code>OAuth2PasswordBearer</code>), <code>tokenUrl = auth/login</code>
Token	JWT assinado, algoritmo <code>HS256</code> (variável <code>ALGORITHM</code>)
Claims	<code>sub</code> = UUID do usuário; <code>exp</code> = emissão + 1 hora (UTC)
Chave	<code>SECRET_KEY</code> do <code>.env</code> — segredo simétrico
Transporte	Header <code>Authorization: Bearer <token></code>
Hash de senha	<code>bcrypt</code> via <code>passlib.CryptContext(schemes=["bcrypt"], deprecated="auto")</code>
Validação	<code>get_current_user()</code> decodifica o JWT e recarrega o usuário do banco a cada requisição
Autorização	Usuário só pode atualizar/excluir a si próprio — comparação <code>current_user.id == user_id</code> , senão <code>403</code>
Refresh token	Não implementado — expirado o token, é necessário novo login
Revogação	Não há blacklist; o token vale até o <code>exp</code>

Erros de autenticação

Código	Detalhe	Condição
<code>401</code>	Token inválido: sub ausente	JWT sem claim <code>sub</code>
<code>401</code>	Token inválido ou expirado	<code>PyJWTError</code> na decodificação
<code>401</code>	E-mail ou senha incorretos	Falha no login
<code>403</code>	Você não tem permissão para...	Tentativa de alterar outro usuário
<code>404</code>	Usuário não encontrado	Token válido cujo <code>sub</code> não existe mais

Pontos de atenção de segurança

- **CORS aberto:** `allow_origins=["*"]` combinado com `allow_credentials=True` — deve ser restringido por domínio antes de ir a produção.
- **Credenciais no código:** usuário e senha do PostgreSQL estão literais em `infrastructure/database.py`.
- **Log de senha em claro:** `get_password_hash()` imprime a senha recebida via `print()` — remover antes de produção.
- **Sem rate limiting** no endpoint de login (exposto a força bruta).
- **HTTPS não é terminado pela aplicação** — deve ser provido por proxy reverso.

7. Especificação dos endpoints

Base local: `http://localhost:8000`. Prefixos por domínio: `/auth`, `/user`, `/profile`, `/assets`.

7.1 Mapa geral

Método	Rota	Auth	Sucesso	Descrição
POST	<code>/auth/login</code>	público	200	Autentica e devolve o access token
POST	<code>/user/register</code>	público	201	Cria uma conta de usuário
PUT	<code>/user/update/{user_id}</code>	JWT	200	Atualiza o próprio cadastro
DEL	<code>/user/delete/{user_id}</code>	JWT	204	Exclui a própria conta (cascata na watchlist)
GET	<code>/profile/watchlist</code>	JWT	200	Lista os ativos acompanhados
POST	<code>/profile/watchlist/add</code>	JWT	200	Adiciona ativos, ignorando duplicatas
POST	<code>/profile/watchlist/remove</code>	JWT	200	Remove ativos da lista
GET	<code>/assets</code>	JWT	200	Busca global de ativos (<code>?search=</code>)
GET	<code>/assets/{ticker}</code>	JWT	200	Cotação e tendência do ativo
GET	<code>/assets/{ticker}/history</code>	JWT	200	Série histórica de fechamento
GET	<code>/assets/{ticker}/financials</code>	JWT	200	Indicadores fundamentalistas
GET	<code>/assets/{ticker}/dividends</code>	JWT	200	Últimos 10 proventos
GET	<code>/assets/{ticker}/news</code>	JWT	200	Até 5 notícias recentes
GET	<code>/docs</code> · <code>/redoc</code> · <code>/openapi.json</code>	público	200	Documentação gerada automaticamente

7.2 Autenticação — `/auth`

POST `/auth/login`

Corpo em `application/x-www-form-urlencoded` (`OAuth2PasswordRequestForm`): o campo `username` recebe o e-mail.

```
Request (form-data)
  username=usuario@exemplo.com
  password=senha-do-usuario

Response 200 (application/json)
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer"
}

Erros
401 { "detail": "E-mail ou senha incorretos" }
```

7.3 Usuário — /user

POST /user/register → 201 Created

```
Request (application/json)
{
  "name": "Felipe Nilo",
  "email": "felipe@exemplo.com",
  "password": "senha-forte",
  "birth_date": "1995-04-21",
  "investor_profile": "MODERATE"
}

Response 201 (UserResponse - sem o hash de senha)
{
  "id": "0f6a1c9e-3b2d-4e57-9c1a-8f2b6d0e7a13",
  "name": "Felipe Nilo",
  "email": "felipe@exemplo.com",
  "birth_date": "1995-04-21",
  "investor_profile": "MODERATE",
  "is_active": true
}

Erros
400 { "detail": "E-mail já registrado." }
422 Falha de validação (e-mail inválido ou perfil fora do enum)
```

PUT /user/update/{user_id} → 200 OK

```
Request (application/json - UserUpdate)
{
  "id": "0f6a1c9e-3b2d-4e57-9c1a-8f2b6d0e7a13",
  "name": "Felipe N. M. de Faria",
  "email": "felipe.novo@exemplo.com",
  "investor_profile": "AGGRESSIVE"
}

Erros
403 { "detail": "Você não tem permissão para atualizar este usuário." }
404 { "detail": "Usuário não encontrado." }
```

DELETE /user/delete/{user_id} → 204 No Content

Sem corpo de resposta. A exclusão remove a watchlist por **ON DELETE CASCADE**.

Erros

```
403 { "detail": "Você não tem permissão para deletar este usuário." }
404 { "detail": "Usuário não encontrado." }
```

7.4 Perfil / Watchlist — /profile

GET /profile/watchlist → List[Asset]

Response 200

```
[
  { "ticker": "PETR4", "name": "Petrobras PN",
    "icon_url": "https://www.google.com/s2/favicons?domain=petrobras.com.br&sz=128" },
  { "ticker": "AAPL", "name": "Apple Inc.",
    "icon_url": "https://www.google.com/s2/favicons?domain=apple.com&sz=128" }
]
```

Usuário sem watchlist retorna [] (lista vazia, não 404).

POST /profile/watchlist/add

Aceita lista de ativos. Duplicatas são descartadas — tanto dentro do payload quanto contra a lista já persistida.

Request (application/json)

```
[
  { "ticker": "PETR4", "name": "Petrobras PN", "icon_url": "https://..." },
  { "ticker": "VALE3", "name": "Vale ON", "icon_url": "https://..." }
]
```

Response 200

```
{ "message": "Ativos processados com sucesso!", "added_count": 2 }
```

Erros

```
400 { "detail": "Todos os ativos já estão na sua lista." }
```

POST /profile/watchlist/remove

Request (application/json — apenas o ticker)

```
[ { "ticker": "PETR4" } ]
```

Response 200

```
{ "message": "Remoção processada com sucesso!", "removed_count": 1 }
```

Erros

```
404 { "detail": "Watchlist não encontrada ou já está vazia." }
404 { "detail": "Nenhum dos ativos informados foi encontrado na sua lista." }
```

7.5 Ativos — /assets

Todo o router exige JWT: dependencies=[Depends(get_current_user)] .

GET /assets?search={termo} → List[Asset]

Sem `search`, a consulta ao Yahoo usa o termo padrão `"P"`. Os ícones de todos os resultados são resolvidos em paralelo via `asyncio.gather`.

GET /assets?search=petro

```
[ { "ticker": "PETR4.SA", "name": "PETROBRAS PN", "icon_url": "https://..." } ]
```

GET /assets/{ticker} → AssetQuote

GET /assets/PETR4

```
{
  "ticker": "PETR4",
  "name": "PETROBRAS PN",
  "icon_url": "https://www.google.com/s2/favicons?domain=petrobras.com.br&sz=128",
  "price_usd": 38.42,
  "direction": "subindo"
}
```

direction ∈ { "subindo", "caindo", "estável" }

Calculado por (price - regularMarketOpen) / regularMarketOpen.

Erros

404 { "detail": "Ativo XPTO não encontrado" }

GET /assets/{ticker}/history?period=1mo → List[HistoryPoint]

Query period (default "1mo") – valores do yfinance:

1d · 5d · 1mo · 3mo · 6mo · 1y · 2y · 5y · 10y · ytd · max

Intervalo fixo: "1d"

```
[ { "date": "2026-08-11", "close": 38.10 },
  { "date": "2026-08-12", "close": 38.42 } ]
```

GET /assets/{ticker}/financials → Financials

```
{
  "market_cap": 512300000000,
  "pe_ratio": 4.87,
  "dividend_yield": 0.1832,
  "target_price": 45.10,
  "recommendation": "buy"
}
```

Todos os campos são opcionais – o Yahoo pode não fornecê-los.

GET /assets/{ticker}/dividends → List[Dividend]

Retorna os 10 proventos mais recentes (.tail(10)).

```
[ { "date": "2026-05-20", "amount": 1.05 } ]
```

GET /assets/{ticker}/news → List[NewsItem]

Limitado às 5 notícias mais recentes.

```
[ {
  "title": "Petrobras anuncia resultados do 2º trimestre",
  "link": "https://finance.yahoo.com/news/...",
  "publisher": "Reuters",
  "summary": "A companhia reportou...",
  "provider_publish_time": "2026-08-12T14:30:00Z"
} ]
```

8. Modelos de dados (schemas Pydantic)

8.1 Domínio de ativos — `models/asset.py`

Schema	Campo	Tipo	Observação
Asset	ticker	str	Código do ativo
	name	str	Nome curto
	icon_url	str	Favicon Google ou Clearbit
AssetRemove	ticker	str	Payload de remoção
AssetQuote (herda Asset)	price_usd	float	Preço corrente
	direction	str	subindo / caindo / estável
HistoryPoint	date	str	ISO YYYY-MM-DD
	close	float	Fechamento
Dividend	date	str	Data do provento
	amount	float	Valor por ação
Financials	market_cap	Optional[int]	Valor de mercado
	pe_ratio	Optional[float]	P/L (trailingPE)
	dividend_yield	Optional[float]	Dividend yield
	target_price	Optional[float]	Preço-alvo médio
	recommendation	Optional[str]	recommendationKey
NewsItem	title	str	Título
	link	str	URL canônica
	publisher	str	Fonte
	summary	Optional[str]	Default ""
	provider_publish_time	str	Data de publicação

8.2 Domínio de usuário — `models/user.py`

Schema	Campos	Uso
UserCreate	name, email: EmailStr, password, birth_date: date, investor_profile: Literal[...]	Entrada de /user/register
UserLogin	email: EmailStr, password	Declarado; o login efetivo usa OAuth2PasswordRequestForm
UserResponse	id: UUID, name, email, birth_date, investor_profile, is_active	Saída pública; from_attributes = True
UserUpdate	id: UUID, name, email: EmailStr, investor_profile: Literal[...]	Entrada de /user/update/{user_id}

`investor_profile` é restrito por `Literal['CONSERVATIVE', 'MODERATE', 'AGGRESSIVE']` na camada Pydantic e por `CHECK CONSTRAINT` na camada de banco — validação em profundidade.

9. Integração externa — Yahoo Finance

9.1 Canais de acesso

Canal	Tecnologia	Uso
Biblioteca <code>yfinance</code>	Síncrona, em threadpool	Cotação, histórico, dividendos, fundamentos, notícias, <code>.info</code>
Endpoint de busca	<code>httpx.AsyncClient</code>	GET <code>https://query1.finance.yahoo.com/v1/finance/search?q={query}</code> , header <code>User-Agent: Mozilla/5.0</code>
Ícones — primário	Google Favicons	<code>https://www.google.com/s2/favicons?domain={dominio}&sz=128</code> , derivado de <code>info["website"]</code>
Ícones — fallback	Clearbit Logo	<code>https://logo.clearbit.com/{ticker}.com</code>

9.2 Normalização automática de ticker

Regras de `YahooFinanceProvider._format_ticker()`, aplicadas após `upper()`:

Condição	Transformação	Exemplo
Comprimento ≥ 5 e último caractere numérico	Sufixo <code>.SA</code> (B3)	<code>PETR4</code> → <code>PETR4.SA</code>
Símbolo em <code>{BTC, ETH, SOL, USDC, USDT}</code>	Sufixo <code>-USD</code>	<code>BTC</code> → <code>BTC-USD</code>
Demais casos	Sem alteração	<code>AAPL</code> → <code>AAPL</code>

Limitação conhecida: a lista de criptomoedas é fixa no código e a regra da B3 é heurística — tickers americanos com 5+ caracteres terminados em dígito seriam incorretamente sufixados com `.SA`.

9.3 Tratamento de concorrência

`yfinance` é totalmente síncrono. Toda chamada é despachada para fora do event loop por `loop.run_in_executor(None, ...)` (provider) ou `run_in_threadpool()` (service), preservando a capacidade de atendimento concorrente do Uvicorn. A propriedade `.info` é a operação mais cara — é ela que motiva o cache e o `asyncio.gather`.

10. Configuração e variáveis de ambiente

Variável	Obrigatória	Default	Descrição
SECRET_KEY	Sim	—	Chave de assinatura do JWT
ALGORITHM	Não	HS256	Algoritmo de assinatura
REDIS_URL	Não	redis://localhost:6379	Em Docker: redis://redis_cache:6379
CACHE_TTL_SECONDS	Não	60	TTL padrão do cache de cotação

Carga feita por `python-dotenv` em `core/config.py`, no import da aplicação.

```
# .env.example
SECRET_KEY=sua_chave_super_secreta_aqui
ALGORITHM=HS256
REDIS_URL=redis://redis_cache:6379
CACHE_TTL_SECONDS=60
```

A conexão com o PostgreSQL **não** é parametrizada por ambiente — está fixa em `infrastructure/database.py`.

11. Implantação e execução

11.1 Topologia de containers

Serviço	Imagem	Porta	Detalhes
api_financeira	build local (python:3.11-slim)	8000:8000	depends_on: redis_cache; env_file: .env; volume ./app; restart: on-failure
redis_cache	redis:alpine	6379:6379	restart: always
PostgreSQL	externo ao compose	5432	Acessado por host.docker.internal — precisa existir no host

Rede Docker: `rede_financeira` (rede default nomeada).

11.2 Build da imagem

O `Dockerfile` usa build em camadas com `uv`: copia `pyproject.toml` e `uv.lock` primeiro, instala dependências e só então copia o código — alterações de código não invalidam a camada de dependências.

```
FROM python:3.11-slim
COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/
WORKDIR /app
COPY pyproject.toml uv.lock* ./
RUN uv pip install --system --no-cache -r pyproject.toml
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

11.3 Comandos operacionais

Comando	Finalidade
<code>docker compose up --build</code>	Sobe API + Redis (requer porta 6379 livre)
<code>uv run uvicorn main:app --reload</code>	Execução local com hot reload
<code>uv add <pacote></code>	Adiciona dependência e atualiza o lockfile
<code>uv sync</code>	Reconstrói o <code>.venv</code> a partir do <code>uv.lock</code>

Pré-requisitos para execução local: Redis em `localhost:6379` e PostgreSQL em `localhost:5432` com a base `easy_finace` criada.

11.4 Middleware CORS

```
CORSMiddleware(  
    allow_origins=["*"],          # restringir em produção  
    allow_credentials=True,  
    allow_methods=["*"],  
    allow_headers=["*"],  
)
```

12. Requisitos não funcionais

Atributo	Especificação atual
Modelo de concorrência	ASGI assíncrono (Uvicorn); I/O bloqueante isolado em threadpool
Latência	<i>Cache hit</i> : milissegundos. <i>Cache miss</i> : dependente do Yahoo Finance (<code>.info</code> é a chamada mais custosa)
Escalabilidade horizontal	Aplicação sem estado — o estado vive no PostgreSQL e no Redis, ambos compartilháveis entre réplicas
Resiliência	Falha na busca de ícone é degradada silenciosamente; <code>get_available_assets</code> retorna <code>[]</code> em erro; containers com política de restart
Observabilidade	Apenas <code>print()</code> — sem logging estruturado, métricas ou tracing
Testes automatizados	Ausentes (existe apenas o script manual <code>teste_logos.py</code>)
Health check	Não implementado
Versionamento da API	Sem prefixo de versão nas rotas (não há <code>/v1</code>)

13. Débitos técnicos identificados

#	Item	Severidade	Recomendação
1	Senha impressa em claro no console (<code>core/security.py</code>)	Alta	Remover os <code>print()</code> de <code>get_password_hash()</code>
2	Credenciais do banco no código-fonte	Alta	Mover para <code>DATABASE_URL</code> no <code>.env</code>
3	CORS * com <code>allow_credentials=True</code>	Alta	Restringir a lista de origens permitidas
4	API Pydantic v1 sob Pydantic 2.12.5 (<code>.dict()</code> , <code>.json()</code> , <code>.parse_raw()</code>)	Média	Migrar para <code>model_dump()</code> , <code>model_dump_json()</code> , <code>model_validate_json()</code> — os métodos antigos estão obsoletos e serão removidos
5	Python 3.11 no container vs. <code>requires-python >=3.12</code>	Média	Alinhar a imagem base para <code>python:3.12-slim</code>
6	Driver de banco síncrono em rotas <code>def</code> ordinárias	Média	Aceitável (FastAPI usa threadpool), mas <code>asyncpg</code> + SQLAlchemy async escala melhor
7	Ausência de migrações de schema	Média	Adotar Alembic
8	Sem testes automatizados	Média	pytest + httpx <code>AsyncClient</code> , com <code>ICacheProvider</code> / <code>IAssetProvider</code> falsos
9	Import não utilizado de <code>curl_cffi</code> em <code>models/user.py</code>	Baixa	Remover
10	<code>added_count</code> em <code>/watchlist/add</code> retorna o total enviado, não o efetivamente inserido	Baixa	Retornar a variável <code>added_count</code> já calculada no laço
11	Sem rate limiting no login	Média	slowapi ou limitação no proxy reverso
12	Campo <code>price_usd</code> nomeia dólar, mas ativos da B3 retornam BRL	Baixa	Renomear para <code>price</code> e incluir campo <code>currency</code>